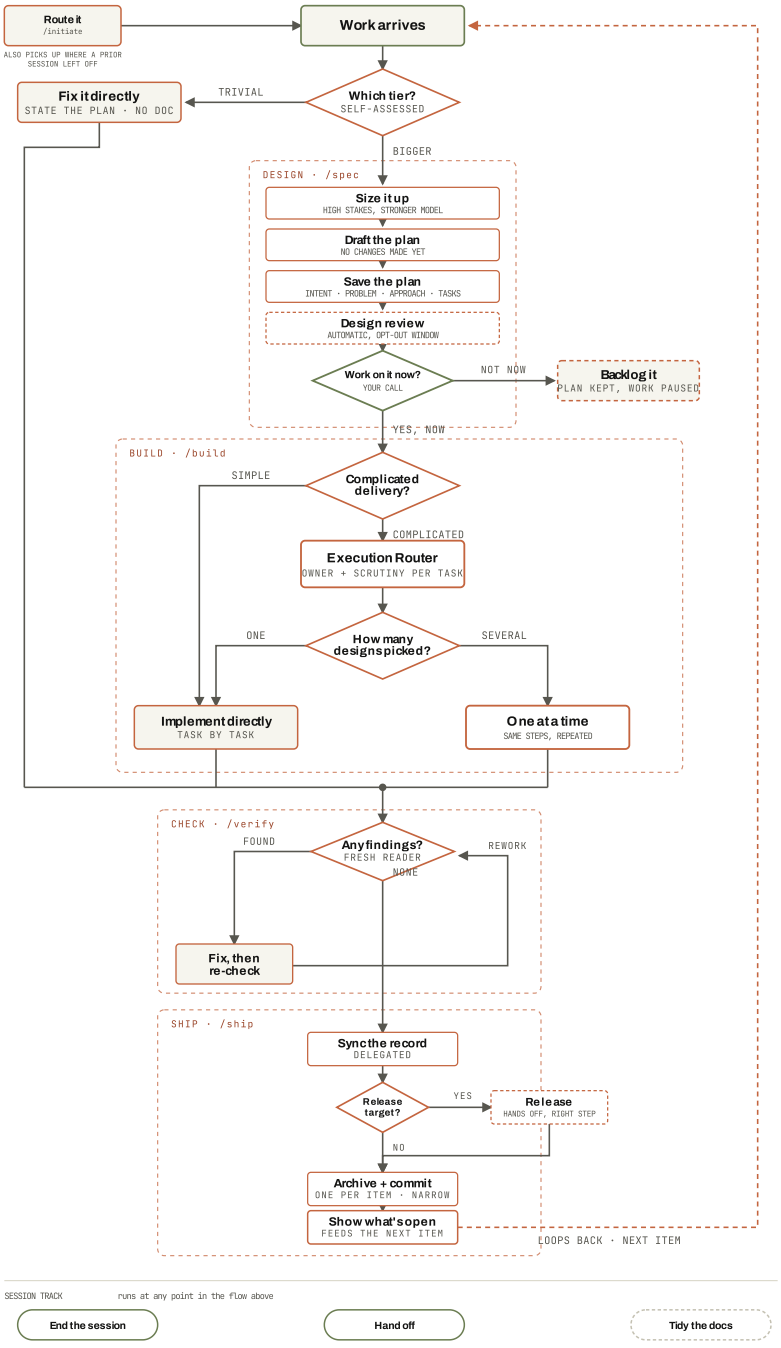
AN AI-NATIVE SDLC, RUN BY ONE PERSON

A Team of One

Teams that build software with AI doing much of the work have converged on a similar shape: plan it, build it, review it, ship it. This process follows that same shape, but reshapes it entirely around one person working alone with an AI assistant instead of a team. The checks a team gets from people in a room, a solo developer gets from a second AI model and a fresh conversation instead, and the whole thing runs itself once it's set up.

Here's the actual decision tree, six forks in total. Two of them end the work with no code written at all, a trivial fix handled on the spot, or a plan set aside rather than built right now. An independent reader looks at the work twice, once at the design review and once at the check, and everything else is either a person deciding something or work simply moving forward.



the assistant checks or acts on its own a person decides here

Every branch is a way of doing **less**, never more — and two of them end the run with no code written at all. The dashed boxes are conditional: the design review fires automatically on the two largest tiers but stops for an opt-out window first, "backlog it" is a real, common outcome rather than a failure state, and the release step is simply absent for work with nothing to deploy.

FIVE IDEAS, IN ORDER

Most AI-assisted work fails one of two ways: either every change gets the same heavy process, or a single design is considered complete without anyone reviewing the work. This is what the process below builds: an AI-centric approach to effective development.

01

No one is a reliable judge of their own work — an AI included

That's the risk behind the second failure mode above — calling a design complete before anyone actually reviews it. An AI assistant reviewing its own output uses the same reasoning that produced it in the first place, so it lands on the same answer. That's true for people too. A developer's own code review only counts for something when someone else actually reads it, and this process builds that same independent second look in on purpose. In a fresh conversation with no memory of how the AI produced the work, a second AI model stands in for the second engineer when there's no person around to do it.

02

How much scrutiny a change gets should match how much a mistake would cost

Not every change deserves the same amount of scrutiny. The AI fixes a one-line issue on the spot, and the day moves on. Something with more at stake — a new feature, or anything expensive to get wrong — earns more scrutiny: a short written plan first, which the AI drafts and a person approves before work starts. The size and the risk of the change decide how much scrutiny it gets — a fixed amount for everything either slows down the small stuff or lets the risky stuff through too fast.

03

That scrutiny happens automatically, at the moments that matter

Deciding how much scrutiny a change deserves only matters if that scrutiny actually happens — automatically, without anyone remembering to ask for it. Four checks make that automatic: a **tier check**, a **design review** before the AI builds anything, an **execution check** confirming the AI is handling the work with the right amount of care, and a **QA check** that has an independent reader confirm the result against what the plan actually specified once the AI finishes. All four fire on their own.

04

"Verified" means checked against something real

None of those checks are worth much if "verified" just means the AI sounded confident. A test that ran, a log the AI read, a number the AI measured: that's what "checked" means here. The single most common failure this catches is the AI reporting work as done on the strength of a confident explanation alone.

05

The AI forgets everything between conversations — so continuity has to be built in

Everything above assumes a round of work happens in one sitting. In practice it often doesn't. A person on a team can pick a half-finished task back up from memory, or a quick conversation with whoever left it. An AI assistant can't — every new conversation starts from zero. So continuity becomes its own explicit step: writing down exactly where things stand, then handing that off to a fresh conversation the same way one person hands off to another on a real team.

> **The same trade scales however big the team gets.** A team buys independent review with a second engineer's time; a team of one buys the same thing with a second AI model and a fresh conversation. Either way, the actual requirement is small — one independent, fresh look before calling something done — which makes it cheap enough to try on the very next thing you build.

Distilled from **Phase-Gate**, the fuller process reference for people actually adopting this day to day, built for solo use. This page explains the idea — and where else it applies — on its own, for anyone who wants the reasoning without the mechanics.